

Spec Kit Training Log & Summary

- **Student Name:** Nguyễn Gia Quốc Uy
 - **Student ID:** 24127261
 - **Class:** 24C11
 - **Author:** Nguyễn Gia Quốc Uy
 - **Editor:** Nguyễn Gia Quốc Uy
 - **Reviewer:** Group 9
-

My Takeaways from the Spec Kit Tutorials

This is my personal summary of what I learned after watching the Spec Kit YouTube tutorials.

The SDD Skill divide a development process into multiple clear phases with the main purpose "think before you code", prevent the fact that AI can make mistake, hallucinate or going off-track.

Some useful information I have collected after watching all the videos on Youtube and research by AI:

1. `constitution.md` (Project Constitution)

- **What it is:** This is a main law of our project. Once our group sets these rules, we cannot change them unless we do a formal update following the rules.
 - **Why it matters:** It stops the AI agent from writing messy code or suddenly switching to a different programming language.
 - **What we put inside:**
 - Our core coding rules (like using `camelCase` for variables and `PascalCase` for UI components).
 - The exact technology stack we agreed to use.
 - How we write Git commit messages and create branches (like `feature/branch-name`).
 - Testing guidelines to make sure our code actually works.
-

2. `spec.md` (Functional Specification - "Specify")

- **What it is:** This file explains **WHAT** we want to build and **WHY** we need it.
 - **The Golden Rule: Absolutely no code or technical details in this file.** It is written purely from the user's perspective.
 - **What we put inside:**
 - **User Stories:** Simple statements to explain the user's need. For example: *As a student, I want to see a Kanban board so that I can track my group tasks.*
 - **Priority Levels:** We group our features into P1, P2, and P3.
 - *P1 (Must-Have):* Core features that the app cannot run without.
 - *P2 (Should-Have):* Important features for a good user experience.
 - *P3 (Could-Have):* Nice-to-have extras, like smooth animations.
 - *Our rule:* We make the AI complete all P1 features first before starting on P2 or P3.
 - **Functional Requirements (FR):** Simple tables listing what the app must do (e.g., *"The system must show an error if the password is too short"*). Each requirement has a unique ID like `REQ-01`.
 - **Out of Scope:** A list of things we will **not** build, so we don't waste time on unnecessary features.
-

3. `plan.md` (Technical Plan - "Plan")

- **What it is:** This is where we figure out **HOW** to build the features we wrote in `spec.md`. It is our technical blueprint.
 - **What we put inside:**
 - The database structure (tables, fields, and how they connect).
 - Our API paths (endpoints, request bodies, and success/error status codes).
 - The exact list of files we need to create or edit.
 - Our reasoning for choosing a specific library or tool over other options.
-

4. `tasks.md` (Technical Blueprint - "Task")

- **What it is:** A step-by-step checklist. The AI reads our specs, technical plans, and project constitution, then automatically generates this file.

- **How it is structured:** It splits the work into logical phases. Each phase has 2-3 specific tasks. Every task shows what code to write, who is doing it, and which tasks must be finished beforehand.
-

5. `clarify.md` (Requirement Clarification - "Clarify")

- **What it is:** A quick Q&A process. The AI reads our `spec.md` and looks for any confusing points or missing details.
 - **How it works:** The AI asks us 3 to 5 questions. We choose the options or write our own answers. The AI then automatically updates the "Clarifications" section at the bottom of our `spec.md` file.
-

6. `analyze.md` (In-Depth Analysis - "Analyze")

- **What it is:** The final sanity check. Before any coding starts, the AI reviews our specs, plans, and tasks.
 - **Goal:** It checks for any mistakes, contradictions, or violations of our project constitution. This helps us find bugs before we even start programming.
-

7. How We Organize Spec Kit Files in Our Repo

Here is how our Group 9 repository (`Software-Engineering- - -Group-9`) is structured to keep these files organized:

```
Software-Engineering---Group-9/ <-- Our project root folder
├── .agents/
│   └── skills/                # Commands for the AI assistant (agy,
copilot, codex,...)
├── .specify/
│   ├── memory/
│   │   └── constitution.md    # Project rules and standards
(Constitution)
│   └── templates/           # Sample templates for spec and plan
files
├── specs/                    # Folder where we organize our features
│   ├── 001-user-login/
│   │   ├── spec.md           # What the login feature does
│   │   ├── plan.md           # How we build the login feature
│   │   └── tasks.md          # Login task checklist
│   └── 002-shopping-cart/
│       ├── spec.md
│       ├── plan.md
│       └── tasks.md
├── src/                      # Our main application code folder
└── README.md
```